Lab 3

Artificial Intelligence Lab

1 2 3 4

Program 1

8-Queens: All Solutions + Board Visualization

Write a Python program to solve the 8-Queens problem using backtracking and print all 92 solutions. Add a board visualization to your solution — print the chessboard using Q for queens and . for empty cells.

eightqueens.py > ...

```python
N = 8
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or \
           board[i] - i == col - row or \
           board[i] + i == col + row:
            return False
    return True
def print_board(board):
    for i in range(N):
        for j in range(N):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
    print()
def solve(row, board):
    global count
    if row == N:
        count += 1
        print(f"Solution {count}:")
        print_board(board)
        return

    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            solve(row + 1, board)
count = 0
board = [-1] * N
solve(0, board)
print("Total solutions:", count)
```

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "
Solution 1:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Solution 2:
Q . . . . . . .
. . . . . Q . .
. . . . . . . Q
. . Q . . . . .
. . . . . . Q .
. . . Q . . . .
. Q . . . . . .
. . . . Q . . .

Solution 3:
Q . . . . . . .
. . . . . . Q .
. . . Q . . . .
. . . . . Q . .
. . . . . . . Q
. Q . . . . . .
. . . . Q . . .
. . Q . . . . .

Solution 4:
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .


Solution 89:
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . Q . . . . .
. . . . . Q . .

Solution 90:
. . . . . . . Q
. Q . . . . . .
. . . . Q . . .
. . Q . . . . .
Q . . . . . . .
. . . . . . Q .
. . . Q . . . .
. . . . . Q . .

Solution 91:
. . . . . . . Q
. . Q . . . . .
Q . . . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . Q . . .
. . . . . . Q .
. . . Q . . . .

Solution 92:
. . . . . . . Q
. . . Q . . . .
Q . . . . . . .
. . Q . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . . . Q .
. . . . Q . . .

Total solutions: 92
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 2

8-Queens: First Valid Solution

Write a Python program to solve the 8-Queens problem to print only the first valid solution found and terminate immediately.

firstvalidsol.py > board

```python
N = 8
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or \
           board[i] - i == col - row or \
           board[i] + i == col + row:
            return False
    return True
def print_board(board):
    for i in range(N):
        for j in range(N):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
def solve(row, board):
    if row == N:
        print("First Solution:")
        print_board(board)
        return True

    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            if solve(row + 1, board):
                return True
    return False
board = [-1] * N
solve(0, board)
```

OUTPUT:-

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desk
First Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 3

Generalized N-Queens Problem

Generalize your solution to solve the N-Queens problem for any N given as input. Test it for N = 4, 6, 8, 10, and 12. Tabulate the number of solutions for each value of N.

nqueens.py > solve_n_queens > solve

```python
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or \
           board[i] - i == col - row or \
           board[i] + i == col + row:
            return False
    return True
def solve_n_queens(N):
    board = [-1] * N
    count = 0

    def solve(row):
        nonlocal count
        if row == N:
            count += 1
            return

        for col in range(N):
            if is_safe(board, row, col):
                board[row] = col
                solve(row + 1)

    solve(0)
    return count
# Testing

for n in [4, 6, 8, 10, 12]:
    print(f"N = {n}, Solutions = {solve_n_queens(n)}")


#N = int(input("Enter value of N: "))

#result = solve_n_queens(N)
#print("Total solutions:", result)
```

OUTPUT:-

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % pytho
N = 4, Solutions = 2
N = 6, Solutions = 4
N = 8, Solutions = 92
N = 10, Solutions = 724
N = 12, Solutions = 14200
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 4 **Pre-Placed Queen Constraint**

Modify the program so that one queen is pre-placed at position (0, 3) — row 0, column 3. How many valid solutions exist with this fixed constraint?

fixedcons.py > ...

```python
N = 8
def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col or \
           board[i] - i == col - row or \
           board[i] + i == col + row:
            return False
    return True
def solve(row, board):
    global count
    if row == N:
        count += 1
        return

    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            solve(row + 1, board)

board = [-1] * N
# Pre-place queen at (0,3)
board[0] = 3
count = 0
solve(1, board)
print("Solutions with queen at (0,3):", count)
```

OUTPUT:-

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % pythc
Solutions with queen at (0,3): 18
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```